Casting Defect Classification

Technical report | 17 October 2026

A CPU-trained binary classifier identifies defective front-view castings. A frozen ImageNet ResNet-18 feature extractor with a trained logistic head achieves 99.30% accuracy and 99.56% defect recall on 715 held-out images. The test set contains two missed defects and three false alarms. The project includes a simpler baseline, validation-based tuning, error analysis, public image-upload inference, and a containerized REST API.

1. Problem and dataset

Image-level folder labels support binary classification: ok_front = 0 and def_front = 1. The public industrial dataset contains 7,348 images at 300 x 300 pixels. Missed defects are the main inspection risk; false alarms create additional manual inspection.

Partition	Acceptable	Defective	Total
Supplied training	2875	3758	6633
Final training	2249	3006	5255
Validation	562	752	1314
Supplied test	262	453	715

The audit found 64 training images identical to test images. Their training copies were removed before a seeded, stratified validation split; all 715 test images were retained. No corrupt files or conflicting exact-duplicate labels were found. Final splits have disjoint decoded-pixel hashes. The related 512 x 512 collection is excluded. Missing physical-part/session identifiers prevent ruling out transformed near duplicates.

2. Models and training

The baseline uses standardized grayscale 32 x 32 pixels and class-weighted logistic regression. The stronger model uses RGB 128 x 128 images, antialiased bilinear resizing, ImageNet normalization, and frozen ResNet-18 features. Its logistic head is trained on original images plus one seeded flipped/brightness-adjusted view per training image. The encoder is not fine-tuned.

Four regularization values per model are compared on validation data. Thresholds maximize validation defect F2; macro F1 selects regularization and model family. Selected C/threshold values are 1/0.25 for the baseline and 0.1/0.50 for ResNet. Test results do not influence selection. Class weights address moderate imbalance. The initial full pipeline took 253.8 seconds with four CPU threads and cached pretrained weights; seed = 42.

3. Evaluation results

Metric	Pixel baseline	ResNet + logistic head
Accuracy	85.73%	99.30%
Balanced accuracy	82.63%	99.21%
Macro F1	0.8391	0.9925
Defect precision	84.89%	99.34%
Defect recall	94.26%	99.56%
Defect F2	0.9222	0.9951
ROC AUC	0.9575	0.9998
Defect average precision	0.9782	0.9999
Brier score	0.07677	0.00488

The selected model correctly predicts 710 of 715 images: 259 acceptable and 451 defective. Its confusion matrix is [[259, 3], [2, 451]], with true labels as rows and acceptable first. The baseline produces 26 missed defects and 76 false alarms.

Stratified bootstrap with 1,000 resamples gives 95% intervals of 98.60% to 99.86% for accuracy, 98.90% to 100.00% for defect recall, and 0.9849 to 0.9985 for macro F1. These intervals assume independent observations within class; related acquisitions could make them too narrow. Probabilities are uncalibrated.

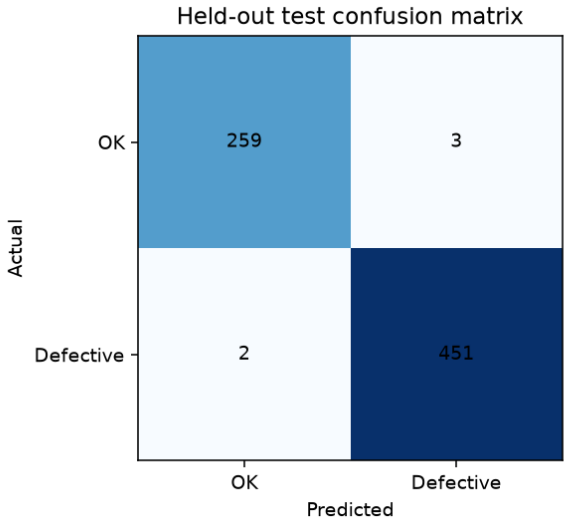


Figure 1. Selected model: held-out test confusion matrix.

4. Error analysis

All five errors are shown below. Visual observations are hypotheses, not expert defect diagnoses. Labels and decision thresholds remain unchanged after test inspection.



Figure 2. All five misclassified test images; 1 = defective, 0 = acceptable.

Image	Defect score	Observation
cast_def_0_2154.jpeg	0.120	Missed defect. A small inner-rim irregularity may be lost at low resolution; the low score also limits uncertainty-only screening.
cast_def_0_1607.jpeg	0.409	Missed defect. Small rim/contour irregularities under strong illumination merit higher-resolution comparison.
cast_ok_0_259.jpeg	0.883	False alarm. Surface texture and diagonal illumination may resemble defect cues.
cast_ok_0_1137.jpeg	0.572	False alarm. Uneven shading and surface marks accompany a score close to the threshold.
cast_ok_0_7839.jpeg	0.572	False alarm. Fine machining texture and pose/lighting differences accompany a borderline positive score.

Four errors occur in the middle contrast third (4/238); one occurs in the low third (1/238), and none in the high third (0/239). Both missed defects occur in the middle brightness third. These descriptive slices do not support a simple extreme-lighting explanation and are too small to establish a stable causal pattern.

5. Deployment and verification

GitHub Pages serves an image-upload interface using ONNX Runtime Web and the exported trained model. RGB conversion and antialiased 128 x 128 resizing precede inference; normalization and the fused scaler/head are inside the graph. Images remain in the browser. Vendored runtime assets eliminate a CDN dependency. Initial model/runtime downloads and browser CPU/memory use are the main deployment costs.

The optional FastAPI service exposes /health and /predict, validates PNG/JPEG uploads, and enforces 10 MiB/20 megapixel limits. Startup verifies the model checksum and performs warm-up. The Docker image runs as a non-root user with a health check and runtime-only dependencies. Authentication and global rate limiting are outside the prototype scope.

Check	Recorded result
Automated tests	16 passed locally; repository automated tests passed.
Public inference	Both classes passed example buttons and actual file uploads; invalid images rejected; no browser/network/HTTP errors.
Artifacts and data	All 7,348 source hashes match the manifest. Model hashes match committed artifacts. All 715 test images reproduce the original confusion matrices.
Metrics and numerical parity	Saved metrics agree within 1e-8; bootstrap intervals reproduce. Maximum ONNX/saved score difference: 7.57e-6; Torch/ONNX: 7.99e-6.
Verification scope	Saved-artifact inference replay; no retraining or clean-environment installation during verification.

Reproduction uses Python 3.12, pinned direct dependencies, config.json, and an external dataset in train/test class folders. The report is generated from saved experiment records. Commands from the repository root:

```
python -m pip install -r requirements-dev.txt
python -m pytest -q
python -m scripts.verify_evaluation --dataset-root DATASET_ROOT
python -m scripts.generate_report
```

6. Limitations and further experiments

The collection covers one part/view and restricted acquisition conditions, without defect-subtype or location annotations. Future experiments include part/session-held-out evaluation, expert-confirmed subtle defects and hard negatives, higher-resolution inputs, augmentation ablations, final-block fine-tuning, probability calibration, and lighting/camera shift evaluation. Detection or segmentation requires location annotations. No test-derived hypothesis has been used to retune the reported model.

References

Dataset: Ravirajsinh Dabhi, casting product image data for quality inspection

Source code and machine-readable evaluation records

Public image-upload inference